oxbow — user guide

oxbow runs FFGL video effect plugins for mixers that can't load plugins themselves. Video leaves your mixer over NDI or OMT, passes through a chain of FFGL effects on oxbow's GPU, and comes back as a new source: a loop in the river.

It exists first for **vMix**, which has no video plugin interface but routes any input to an NDI output and (since vMix 29) speaks OMT natively. It works just as well with anything else that can send and receive NDI or OMT.

What you need

- **The oxbow binary** for your platform (macOS or Windows x64).
- **At least one transport runtime**, found at run time, never bundled:
  - *NDI*: install the NDI runtime or NDI Tools from ndi.video.
  - *OMT*: download the binaries release from github.com/openmediatransport/libomtnet and put `libomt`, `libvmx` (and `libomtnet.dll`) together somewhere on the library path — or beside a directory you point `DYLD_LIBRARY_PATH` (macOS) / `PATH` (Windows) at.
  - A transport that isn't installed just reports itself unavailable, and the other keeps working.
- **FFGL 2.x plugins**: `.dll` on Windows, `.bundle` on macOS. Any plugin that loads in Resolume Arena/Avenue should load here.

The **input** is always NDI or OMT, since those are the two things a mixer can send you. The **output** has more choices, because by then the frame is already on oxbow's GPU. See "Where the output goes" below.

Quick start (vMix)

1. In vMix, route the input you want processed to an output: *Settings → Outputs / NDI / OMT*, set e.g. **Output 3** to that input, enabled as NDI or OMT.
2. Write a config, `oxbow.json`:

```
{
  "input":  { "protocol": "ndi", "source": "Output 3" },
  "output": { "protocol": "omt", "name": "oxbow" },
  "control": { "port": 8720 },
  "chain": [
    { "plugin": "C:/Program Files/Common Files/FFGL/Effect.dll",
      "params": { "Mix": 0.8 } }
  ]
}
```

`input.source` is matched case-insensitively as a substring of the full source name (`"HOST (Output 3)"`), so the short form is enough.

3. Run it:

```
oxbow run --config oxbow.json
```

4. Add the processed source back into vMix: *Add Input → NDI / OMT* and pick `oxbow`. Open `http://127.0.0.1:8720/` for live parameter control.

Run oxbow on the vMix machine or on a second machine on the same network. The second machine keeps the GPU load away from vMix and only costs network transport.

Where the output goes

`output.protocol` in the config, or `--out-proto` on the command line. The input side is unchanged (NDI or OMT), but the processed frame can leave by any of five routes, and which one you want is mostly a question of *where the receiving application is*.

Protocol	Platform	Use it when
`ndi`	both	The receiver is on another machine, or is anything on the network
`omt`	both	Same, and both ends speak OMT — vMix 29 and newer do
`syphon`	macOS only	The receiver is **on this Mac**
`spout`	Windows only	The receiver is **on this PC**
`decklink`	both, self-built	The picture has to leave as SDI or HDMI

Syphon and Spout are the same idea under two names: the frame is handed over as a shared GPU surface, so it never leaves the graphics card, is never compressed, and adds no network hop. On one machine they're strictly better than NDI: lower latency, no bandwidth, no encode artefacts. They're also video-only and single-machine, and that's the trade. I've named them for the protocol rather than calling both "shared" on purpose, because that's what the *other* application calls it, and asking for the wrong platform's one answers "syphon output is macOS only" rather than "unknown protocol".

```
{ "output": { "protocol": "syphon", "name": "oxbow" } }
```

DeckLink puts the chain's output on an SDI or HDMI connector. `name` picks the device: a decimal index (`"0"` , `"1"`) or a substring of the card's name (`"Duo"` , `"UltraStudio"`); empty takes the first card, which is right when there's only one. The display mode is chosen from the frame: oxbow looks for a mode matching the incoming size and rate, and refuses rather than guessing if the card won't carry it as 8-bit BGRA.

Two things to know before you plan around it:

- **The published downloads don't have it.** DeckLink support needs Blackmagic's SDK, whose licence isn't mine to redistribute, so it's compiled out unless you build with `-DDECKLINK_SDK_DIR=...` . A build without it tells you so instead of failing in some obscure way.
- **No audio on the card.** oxbow passes audio through its NDI/OMT output untouched, but the DeckLink path is video only. Keep the audio in the mixer.

This path *has* been run against real hardware: a Duo 2, NDI in, an effect on the GPU, SDI out at 60.0 fps.

Commands

```
oxbow probe <plugin>                 plugin metadata and parameters
oxbow selftest <plugin> [--set N=V]  offscreen render test, PASS/FAIL
oxbow list [--proto ndi|omt]         discover sources
oxbow run --config <file.json>       the loop (or --in/--out/--plugin flags)
          [--out-proto ndi|omt|syphon|spout|decklink]
oxbow send-test [--proto ndi|omt|syphon|spout|decklink]
                                     built-in 720p60 test pattern source
oxbow recv-probe --in <source>       receive + fingerprint frames
          [--proto ndi|omt] [--dump frame.ppm]
```

`send-test` and `recv-probe` mean you can test the whole loop with no mixer at all: pattern out, through the chain, probed and dumped at the far end.

`probe` **reports more than a name and a range.** It prints each parameter's `group=` (the section the plugin's author filed it under) and, for dropdown parameters, one line per option. That matters for building a control surface: Porthole's Quality parameter *declares* a range of 0..1 while its options run 0 to 4, so anything that trusted the range and drew a slider was wrong. Read the elements, not the range.

The control page and API

With `"control": { "port": 8720 }` (or `--port`), oxbow serves a page on loopback: status, the effect chain, and a slider for every parameter, applied between frames. The API underneath is three endpoints:

```
GET  /api/state
POST /api/param?effect=0&name=Mix&value=0.5
GET  /api/sources?proto=ndi|omt
```

Anything that can issue HTTP (Companion, a script, a cron job) can drive parameters. Set `"bind"` to expose the page beyond the machine, knowing what that means.

Latency

Expect the loop to add roughly 3–6 frames end to end: transport encode and decode on each leg plus one frame of processing. Running at 50/60p halves the wall-clock cost. Keep the source's audio inside the mixer and delay it to match, or take the loop's audio back, since oxbow passes audio through untouched.

Troubleshooting

- **"connects but no frames arrive" over OMT**: look for a lingering oxbow (or other OMT sender) holding the listen port: `lsof -iTCP:6400` (macOS) / `netstat -ano | findstr 6400` (Windows), and kill it. Old processes can survive Ctrl-C if they wedged before the signal handlers were in place.
- **OMT internals**: set `OXBOW_OMT_LOG=1` for libomt's own log on stderr.
- **A plugin fails to instantiate**: run `oxbow selftest <plugin>`. If the log callback prints nothing and instantiate still fails, the plugin may reject a text-parameter default set during instantiation (a stock-SDK bug pattern); ask the plugin's author about `SetTextParameter`.
- **A DeckLink output keeps transmitting after oxbow exits.** That's the card, not oxbow. A DeckLink output goes on sending valid, locked black long after the process that opened it has gone: measured twenty seconds after the transmitter was killed, a looped-back input still reported a good, locked 1080p50 signal. So killing oxbow **doesn't** test what a downstream device does when the signal disappears; pull the cable, or probe a connector with nothing plugged into it.
- `syphon` / `spout` **reports "macOS only" or "Windows only".** The command line came from the other platform's documentation. Each is that platform's name for the same shared-surface handover; use the local one.
- `decklink` **says the build has no DeckLink support.** It's compiled out unless configured with `-DDECKLINK_SDK_DIR`, and the published downloads are built without it. Build it yourself against Blackmagic's SDK.
- **Logs**: a rotating log lives in the platform log directory (`~/Library/Logs/oxbow` on macOS, `%LOCALAPPDATA%\oxbow\logs` on Windows); `OXBOW_LOG=debug` raises the level.